

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: EC2 Instance Cannot Access S3 Due to IAM Permission Issues

Interview Question

An EC2 instance needs to access an S3 bucket, but the application receives an "Access Denied" error. How would you troubleshoot it?

Answer

If an EC2 instance cannot access an S3 bucket, I first verify the IAM permissions and bucket configuration.

Step 1: Check the IAM Role

Ensure the EC2 instance has an **IAM Role** attached.

If no role is attached, the instance cannot access AWS services securely.

Step 2: Verify IAM Permissions

Check whether the IAM Role has the required permissions.

Example:

- `s3:GetObject`
 - `s3:ListBucket`
 - `s3:PutObject` (if upload is required)
-

Step 3: Check the S3 Bucket Policy

Ensure the bucket policy allows access to the IAM Role.

A bucket policy can override IAM permissions and deny access.

Step 4: Test Access

Run:

```
aws s3 ls s3://my-bucket
```

If access is denied, review the IAM policy and bucket policy.

Step 5: Check for Explicit Deny

Verify there are no:

- Explicit **Deny** statements in IAM policies.
 - Bucket policy denies.
 - Organization **Service Control Policies (SCPs)** blocking access.
-

Real-Time Scenario

A Jenkins server on EC2 couldn't upload build artifacts to S3. The IAM Role had `GetObject` permission but was missing `PutObject`. After updating the IAM policy, uploads succeeded without using access keys.

Best Practices

- Use **IAM Roles** instead of access keys.
 - Grant only the required permissions (least privilege).
 - Regularly review IAM and bucket policies.
 - Avoid using wildcard (*) permissions.
-

Interview Summary

"If an EC2 instance gets an 'Access Denied' error for S3, I first verify the IAM Role attached to the instance, check the IAM policy, review the S3 bucket policy, test access using the AWS CLI, and ensure there are no explicit Deny statements from IAM, bucket policies, or SCPs."

Scenario 2: User Cannot Access AWS Console After Permission Changes

Interview Question

A user reports that they can log in to the AWS Console but receive "Access Denied" when trying to access AWS services. How would you troubleshoot it?

Answer

If a user gets an **Access Denied** error after logging in, I verify the IAM permissions and any policies that may be restricting access.

Step 1: Check IAM Policies

Verify that the correct IAM policies are attached to the user, group, or IAM role.

Ensure the required permissions are present.

Step 2: Check for Explicit Deny

Look for **Explicit Deny** in:

- IAM Policies
- Permission Boundaries
- Service Control Policies (SCPs)
- Resource-based Policies

An explicit **Deny** always overrides an **Allow**.

Step 3: Verify Group Membership

Ensure the user belongs to the correct IAM group and hasn't been removed accidentally.

Step 4: Use IAM Policy Simulator

Use the **IAM Policy Simulator** to identify which policy is denying access.

This helps pinpoint the exact permission issue.

Step 5: Review CloudTrail

Check **CloudTrail** logs for authorization failures to identify the denied action and the policy responsible.

Real-Time Scenario

A developer lost access to EC2 after a policy update. Investigation showed a newly attached **Permission Boundary** was blocking `ec2:*` actions. After updating the boundary, access was restored.

Best Practices

- Assign permissions through **IAM Groups** or **IAM Roles** instead of individual users.
 - Follow the **Principle of Least Privilege**.
 - Test policy changes before applying them in production.
 - Enable CloudTrail for auditing.
-

Interview Summary

"If a user receives an 'Access Denied' error, I check the attached IAM policies, verify there are no explicit Deny statements from Permission Boundaries, SCPs, or resource policies, confirm the user's group membership, use the IAM Policy Simulator to identify the issue, and review CloudTrail logs for failed authorization events."

Scenario 3: Cross-Account IAM Role Assumption Fails

Interview Question

An application in AWS Account A needs to assume an IAM Role in Account B, but it receives an "Access Denied" error. How would you troubleshoot it?

Answer

If cross-account role assumption fails, I verify the trust relationship, IAM permissions, and role configuration.

Step 1: Check Trust Policy

Ensure the IAM Role in **Account B** trusts **Account A** or the specific IAM Role.

Without a valid trust policy, `AssumeRole` will fail.

Step 2: Verify IAM Permissions

Ensure the user or IAM Role in **Account A** has permission to perform:

```
sts:AssumeRole
```

on the target role.

Step 3: Verify Role ARN

Confirm the application is using the correct **IAM Role ARN**.

An incorrect ARN results in an access error.

Step 4: Test Role Assumption

Use the AWS CLI to verify:

```
aws sts assume-role \
--role-arn arn:aws:iam::<account-id>:role/<role-name> \
--role-session-name TestSession
```

If it fails, review the error message to identify whether it's a trust policy or permission issue.

Step 5: Check CloudTrail

Review **CloudTrail** for failed `AssumeRole` events to identify the exact reason for the denial.

Real-Time Scenario

A Jenkins server in **Account A** needed to deploy resources in **Account B**. The IAM Role had the correct permissions, but the **Trust Policy** did not trust the Jenkins IAM Role. After updating the Trust Policy, the pipeline successfully assumed the role and completed the deployment.

Best Practices

- Use IAM Roles instead of long-term access keys.
 - Grant only the required `sts:AssumeRole` permission.
 - Keep Trust Policies as restrictive as possible.
 - Monitor `AssumeRole` events using CloudTrail.
-

Interview Summary

"If cross-account role assumption fails, I first verify the Trust Policy in the target account, ensure the source IAM Role has `sts:AssumeRole` permission, confirm the correct Role ARN is being used, test the role assumption with the AWS CLI, and review CloudTrail logs for authorization failures."

Scenario 4: Lambda Function Cannot Access DynamoDB or S3

Interview Question

Your Lambda function needs to access DynamoDB or S3, but it fails with an "Access Denied" error. How would you troubleshoot it?

Answer

If a Lambda function cannot access AWS services, I verify the execution role and resource permissions.

Step 1: Check the Lambda Execution Role

Ensure the Lambda function has the correct **Execution Role** attached.

Without an IAM role, Lambda cannot access AWS services.

Step 2: Verify IAM Permissions

Ensure the execution role has the required permissions.

Examples:

- `dynamodb:GetItem`
 - `dynamodb:PutItem`
 - `s3:GetObject`
 - `s3:PutObject`
-

Step 3: Check Resource Policies

If accessing S3, verify the **Bucket Policy** allows access from the Lambda execution role.

If using KMS encryption, ensure the role has **KMS decrypt** permissions.

Step 4: Review CloudWatch Logs

Check the Lambda logs in **CloudWatch** for detailed error messages such as:

- `AccessDenied`

- UnauthorizedOperation
 - KMS Access Denied
-

Step 5: Test the Function

Run a test event and confirm the Lambda function can successfully access the required AWS service.

Real-Time Scenario

A Lambda function was unable to upload files to an S3 bucket. Investigation showed the execution role had **GetObject** permission but was missing **PutObject**. After updating the IAM policy, the uploads completed successfully.

Best Practices

- Use dedicated IAM roles for each Lambda function.
 - Follow the **Principle of Least Privilege**.
 - Enable CloudWatch Logs for troubleshooting.
 - Avoid using wildcard (*) permissions.
-

Interview Summary

"If a Lambda function cannot access DynamoDB or S3, I first verify the execution role, check the IAM permissions, review the resource policy and KMS permissions if applicable, analyze CloudWatch logs for authorization errors, and test the function after making the required changes."

Scenario 5: ECS Task Cannot Access AWS Services

Interview Question

Your application is running as an ECS task, but it cannot access AWS services like S3, SQS, or Secrets Manager. How would you troubleshoot it?

Answer

If an ECS task cannot access AWS services, I verify the IAM roles, task configuration, and permissions.

Step 1: Check the Task IAM Role

Ensure the ECS Task has the correct **Task IAM Role** attached.

Note: Use the **Task Role** for application access to AWS services, not the **Execution Role**.

Step 2: Verify IAM Permissions

Ensure the Task Role has the required permissions.

Examples:

- `s3:GetObject`
 - `sqs:SendMessage`
 - `secretsmanager:GetSecretValue`
-

Step 3: Check the ECS Task Definition

Verify the correct **Task Role ARN** is specified in the Task Definition.

If the wrong role is attached, the application won't receive the required permissions.

Step 4: Check Resource Policies

If accessing S3 or Secrets Manager, verify the resource policy allows access from the ECS Task Role.

Step 5: Review Logs

Check:

- ECS Task Logs (CloudWatch)
- Application logs

Look for errors such as:

- `AccessDenied`
 - `UnauthorizedOperation`
-

Real-Time Scenario

An ECS application couldn't retrieve database credentials from **AWS Secrets Manager**. Investigation showed the **Task Role** lacked `secretsmanager:GetSecretValue` permission. After updating the IAM policy, the application started successfully.

Best Practices

- Use **Task Roles** instead of storing AWS credentials.
 - Keep **Execution Role** and **Task Role** separate.
 - Grant only the required permissions.
 - Monitor ECS logs using CloudWatch.
-

Interview Summary

"If an ECS task cannot access AWS services, I first verify the Task IAM Role, check the required IAM permissions, confirm the correct Task Role is attached in the Task Definition, review any resource policies, and analyze CloudWatch logs for authorization errors."

Scenario 6: EKS Pod Cannot Access S3 (IRSA Issue)

Interview Question

An application running inside an EKS Pod needs to access an S3 bucket, but it receives an "Access Denied" error. How would you troubleshoot it?

Answer

If an EKS Pod cannot access AWS services using **IAM Roles for Service Accounts (IRSA)**, I verify the IAM role, service account, and OIDC configuration.

Step 1: Check the Service Account

Verify the Pod is using the correct Kubernetes **Service Account**.

```
kubectl get pod <pod-name> -o yaml
```

Step 2: Verify IAM Role Annotation

Ensure the Service Account is annotated with the correct IAM Role ARN.

```
kubectl describe sa <service-account>
```

Step 3: Check IAM Permissions

Ensure the IAM Role has the required permissions.

Example:

- `s3:GetObject`
 - `s3:ListBucket`
-

Step 4: Verify OIDC Provider

Confirm the EKS cluster has an **OIDC Provider** configured and the IAM Role's **Trust Policy** trusts that OIDC provider.

Step 5: Check Logs

Review:

- Pod logs
- CloudTrail
- Application logs

Look for `AccessDenied` or authentication errors.

Real-Time Scenario

An application running on EKS couldn't read files from an S3 bucket. The IAM policy was correct, but the **Service Account was missing the IAM Role annotation**. After updating the Service Account and restarting the Pod, access was restored.

Best Practices

- Use **IRSA** instead of storing AWS credentials in Pods.
 - Create separate IAM Roles for different applications.
 - Follow the principle of least privilege.
 - Monitor access using CloudTrail.
-

Interview Summary

"If an EKS Pod cannot access S3, I verify the Kubernetes Service Account, IAM Role annotation, IAM permissions, OIDC provider, and Trust Policy. Finally, I review Pod logs and CloudTrail to identify the root cause."

Scenario 7: Temporary AWS Credentials Have Expired (STS Token Expired)

Interview Question

Your Jenkins pipeline or application suddenly fails with "ExpiredToken" or "The security token included in the request is expired." How would you troubleshoot it?

Answer

If I receive an **ExpiredToken** error, I verify the temporary credentials and how they are being generated.

Step 1: Identify the Credential Type

Check whether the application is using:

- IAM Role
- STS AssumeRole
- Temporary session credentials

Temporary credentials expire automatically.

Step 2: Verify Credential Expiry

Check whether the STS session has expired.

```
aws sts get-caller-identity
```

If it fails with `ExpiredToken`, new credentials are required.

Step 3: Refresh Credentials

If using:

- **EC2 IAM Role** → Refresh the session automatically.
 - **Jenkins/GitHub Actions** → Re-run the `AssumeRole` step to generate new temporary credentials.
-

Step 4: Check the Pipeline

Verify the CI/CD pipeline isn't storing expired credentials in environment variables or configuration files.

Always generate fresh credentials at runtime.

Step 5: Review CloudTrail

Check CloudTrail for `AssumeRole` or `ExpiredToken` events to confirm the failure reason.

Real-Time Scenario

A Jenkins deployment pipeline failed with **ExpiredToken** because it reused STS credentials generated several hours earlier. The pipeline was updated to call **AssumeRole** before every deployment, ensuring fresh credentials for each run.

Best Practices

- Use IAM Roles instead of long-term access keys.
 - Generate fresh STS credentials for every pipeline execution.
 - Never hardcode temporary credentials.
 - Monitor `AssumeRole` activity using CloudTrail.
-

Interview Summary

"If I encounter an `ExpiredToken` error, I first identify whether the application is using temporary credentials, verify the session expiry, regenerate credentials using `AssumeRole` or the IAM Role, ensure the CI/CD pipeline doesn't cache expired credentials, and confirm the issue using CloudTrail logs."

Scenario 8: IAM Policy Changes Are Not Taking Effect

Interview Question

You updated an IAM policy, but the user or application still gets "Access Denied." How would you troubleshoot it?

Answer

If IAM policy changes don't take effect, I verify all policies that influence access.

Step 1: Verify the Policy is Attached

Ensure the updated IAM policy is attached to the correct:

- User
 - Group
 - Role
-

Step 2: Check for Explicit Deny

Look for **Explicit Deny** in:

- IAM Policies
- Permission Boundaries
- Service Control Policies (SCPs)
- Resource-based Policies (e.g., S3 Bucket Policy)

An Explicit Deny always overrides an Allow.

Step 3: Verify the Resource ARN

Ensure the policy references the correct resource ARN.

A wrong bucket name, role ARN, or resource ARN can cause access failures.

Step 4: Use IAM Policy Simulator

Use the **IAM Policy Simulator** to determine which policy is allowing or denying the request.

Step 5: Review CloudTrail

Check CloudTrail logs for **AccessDenied** events to identify the exact permission that is failing.

Real-Time Scenario

A developer was granted permission to access an S3 bucket but still received **Access Denied**. Investigation showed the IAM policy was correct, but the **S3 Bucket Policy contained an Explicit Deny**. After updating the bucket policy, access was restored.

Best Practices

- Follow the **Principle of Least Privilege**.
 - Test IAM policies before deploying them.
 - Avoid unnecessary Explicit Deny statements.
 - Use IAM Policy Simulator for troubleshooting.
-

Interview Summary

"If IAM policy changes don't take effect, I verify the policy attachment, check for **Explicit Deny** in IAM, SCPs, Permission Boundaries, and resource policies, validate the resource ARN, use the IAM Policy Simulator, and review CloudTrail logs to identify the root cause."
